

MAIE 5102

Fundamentals of Artificial Intelligence

Final Exam Complete Notes

目录

Lecture 1: Introduction to Artificial Intelligence	9
1 What is AI? ——四种定义	9
2 Rational Agent (理性智能体)	9
3 Agent 与 Environment	10
4 Environment 分类	10
5 Search 与 Rationality 的关系	11
6 MDP ——Markov Decision Process	11
7 Bellman Equation (贝尔曼方程)	12
8 Q-learning (强化学习)	13
9 CSP (约束满足问题)	13
10 Lecture 1 核心逻辑	14
11 考试真正核心	14
Lecture 2: Linear Regression and Classification	15
12 Machine Learning Overview	15
13 Linear Regression (线性回归)	15

目录	2
14 Linear Classification (线性分类)	16
15 Regression vs Classification 对比	17
Lecture 3: Linear Classifier for Image Classification	18
16 Image Classification Problem (图像分类问题)	18
17 Nearest Neighbor (最近邻)	18
18 Multi-class Linear Classifier (多类线性分类器)	19
19 Non-linear Features (非线性特征映射)	19
20 KNN vs Linear Classifier 对比	19
21 Hyperparameter 选择与 Cross-Validation	20
Lecture 4: Neural Networks and Backpropagation	21
22 从线性到非线性: 为什么需要神经网络	21
23 Neural Network Structure (神经网络结构)	21
24 Activation Functions (激活函数)	21
25 Backpropagation (反向传播)	22
26 Computational Graph 中的梯度传播规则	22
27 2-Layer Network 训练示例	22
28 Vanishing Gradient Problem (梯度消失)	22
Lecture 5: Unsupervised Learning and Generalization	24
29 K-means Clustering (K 均值聚类)	24
30 SGD (随机梯度下降)	25
31 Overfitting 与 Generalization (过拟合与泛化)	25
Lecture 6: Uninformed Search	26
32 Search Problem 定义	26

目录	3
33 Tree Search vs Graph Search	26
34 搜索算法详解	26
35 算法对比总结	28
36 UCS 手动展开示例	28
Lecture 7: Informed Search	29
37 Informed Search 的动机	29
38 Greedy Best-First Search (贪心最佳优先搜索)	29
39 A* Search	29
40 Admissibility (可容许性)	29
41 Consistency (一致性 / 单调性)	30
42 Heuristic 设计方法	30
43 Heuristic 的组合与比较	30
Lecture 8: Constraint Satisfaction Problems	32
44 CSP 定义	32
45 Backtracking Search (回溯搜索)	32
46 Variable Ordering: MRV	32
47 Value Ordering: LCV	32
48 Forward Checking (前向检查)	33
49 Arc Consistency (弧一致性)	33
50 Forward Checking vs Arc Consistency 对比	34
51 Min-Conflicts (最小冲突, 局部搜索)	34
Lecture 9: Local Search Algorithms	35
52 Local Search 的特点	35

目录	4
53 Hill Climbing (爬山法)	35
54 Simulated Annealing (模拟退火)	35
55 Genetic Algorithm (遗传算法)	36
Lecture 10: Markov Decision Process I	37
56 从 Search 到 MDP	37
57 MDP 定义	37
58 Discount Factor (折扣因子)	37
59 Policy (策略)	38
60 Value Function (价值函数)	38
61 Bellman Optimality Equation (贝尔曼最优方程)	38
62 Value Iteration (值迭代)	38
Lecture 11: Markov Decision Process II	40
63 Policy Iteration (策略迭代)	40
64 Value Iteration vs Policy Iteration	41
65 Q-Value 函数	41
Lecture 12: Reinforcement Learning I	42
66 RL vs Planning 的区别	42
67 Model-Based RL (基于模型的 RL)	42
68 Model-Free RL (无模型 RL)	42
69 Temporal Difference Learning (TD 学习)	43
Lecture 13: Reinforcement Learning II	44
70 Q-Learning	44
71 Exploration vs Exploitation	44

目录	5
72 SARSA (On-policy TD Control)	45
73 Approximate Q-Learning (函数逼近 Q 学习)	45
74 Deep Q-Network (DQN)	45
Lecture 14: Games I	47
75 Game 的分类	47
76 Minimax (极小极大)	47
77 Depth-Limited Minimax (有限深度 Minimax)	48
78 Minimax 手动计算示例	48
Lecture 15: Games II —Alpha-Beta Pruning and Expectiminimax	49
79 Alpha-Beta Pruning (Alpha-Beta 剪枝)	49
80 Alpha-Beta 手动计算示例	50
81 Expectiminimax (含随机性的博弈)	50
Lecture 16: Probabilistic Reasoning I —Uncertainty	52
82 为什么需要概率	52
83 核心概率规则	52
84 Bayes' Rule (贝叶斯规则)	52
85 Bayes' Rule 计算示例	53
86 Independence 与 Conditional Independence	53
87 从 Joint Distribution 提取 Conditional	53
Lecture 17: Probabilistic Reasoning II —Bayes Net Representation	55
88 联合分布的参数爆炸	55
89 Bayesian Network (贝叶斯网络)	55
90 D-Separation (方向分离)	55

目录	6
91 BN 的构建原则	56
Lecture 18: Probabilistic Reasoning III —Bayes Net Inference	57
92 推理问题形式化	57
93 Inference by Enumeration (枚举推理)	57
94 Variable Elimination (VE) (变量消除)	57
95 Factor 操作详解	58
96 VE 计算示例	58
97 Elimination Order (消去顺序)	58
Lecture 19: Naive Bayes Classifier	59
98 Naive Bayes 的核心假设	59
99 分类规则	59
100 参数估计	59
101 Laplace Smoothing (拉普拉斯平滑)	59
102 Naive Bayes 完整计算示例	60
Lecture 20: Logic Reasoning	61
103 Propositional Logic (命题逻辑)	61
104 First-Order Logic (FOL) (一阶逻辑)	61
105 FOL 翻译的核心规则	62
106 量词的否定	62
107 常见错误分析	62
108 FOL Inference 简述	63
Lecture 21: Applications —Visual Segmentation	64
109 语义分割 (Semantic Segmentation)	64

目录	7
110 上采样方法	64
111 评估指标——mIoU	64
112 实例分割 (Instance Segmentation)	64
Lecture 22: Applications —Object Detection	66
113 Object Detection 任务	66
114 Two-Stage Detectors (两阶段检测器)	66
115 One-Stage Detectors (单阶段检测器)	66
116 关键概念	67
Lecture 23: Self-Supervised Representation Learning	68
117 Self-Supervised Learning 概述	68
118 Pretext Tasks (代理任务)	68
119 Contrastive Learning (对比学习)	68
120 CLIP (Contrastive Language-Image Pre-training)	68
Lecture 24: Visual-Language Pretraining for Open World	70
121 Open World Problems	70
122 CLIP 应用于 Open World	70
123 Depthwise Separable Convolution (深度可分离卷积)	71
A 答题模板 (Answer Templates)	72
A.1 Problem 1 类型: First-Order Logic 翻译	72
A.1.1 题型识别	72
A.1.2 核心规则速记	72
A.1.3 样卷题目复现与答题步骤	72
A.1.4 常见错误	73
A.2 Problem 2 类型: Markov Decision Process (MDP)	73
A.2.1 题型识别	73
A.2.2 Part (i): 计算期望即时奖励	73

A.2.3	Part (ii): Value Iteration 计算	74
A.2.4	Part (iii): 最优策略提取	75
A.2.5	Gridworld 方向辨认 (样卷坐标系)	76
B	关键词识别与考点速查	76
B.1	FOL 翻译关键词对照表	76
B.2	MDP 计算步骤速查	77
C	核心公式速查表	77
C.1	First-Order Logic	77
C.2	Markov Decision Process	77
C.2.1	基本定义	77
C.2.2	Value Iteration	78
C.2.3	最优策略提取	78
C.2.4	Gridworld 转移概率模板	78
D	易错点与考试提醒	78
D.1	FOL 翻译	78
D.2	MDP 计算	78
D.3	考试时间分配建议	79

Lecture 1: Introduction to Artificial Intelligence

1 What is AI? ——四种定义

AI 的四种经典定义 (Russell & Norvig):

	Human-like	Rational
Thinking	Think like people	Think rationally
Acting	Act like people	Act rationally

解释:

- Think like people: 研究人类思维机制 (认知科学)
- Act like people: 通过 Turing Test
- Think rationally: 基于逻辑推理
- Act rationally: 最大化期望性能

考试通常不考历史, 而考: "现代 AI 主要采用哪种定义?"

答案:

Act Rationally

2 Rational Agent (理性智能体)

定义:

An agent that acts to maximize expected performance.

数学表达:

$$a^* = \arg \max_a \mathbb{E}[Performance]$$

逐符号解释:

- a^* : 最优动作
- $\arg \max$: 选取使表达式最大的动作
- $\mathbb{E}$: 期望 (因为环境可能有不确定性)

考试如何体现?

- UCS 为什么 optimal?
- Bellman 方程为什么有 $\max_a$?
- Q-learning 为什么取最大 Q?

本质都来自：

maximize expected performance

3 Agent 与 Environment

Agent 通过：

- Sensors 感知环境
- Actuators 作用环境

形式化表示：

$$f : P^* \rightarrow A$$

含义：

- 输入：所有历史感知
- 输出：动作

Policy 表达：

$$a_t = \pi(s_t)$$

(在 MDP 假设下只依赖当前状态)

4 Environment 分类

五个维度：

- Fully observable vs Partially observable
- Deterministic vs Stochastic
- Episodic vs Sequential

- Static vs Dynamic
- Discrete vs Continuous

确定性环境:

$$s_{t+1} = f(s_t, a_t)$$

随机环境:

$$P(s_{t+1}|s_t, a_t)$$

你的 Search 样题默认是 deterministic。

因为:

每个 action 只有一个 successor。

5 Search 与 Rationality 的关系

UCS 规则:

Expand node with smallest $g(n)$

其中:

$$g(n) = \text{path cost}$$

为什么 optimal?

因为它实现:

minimize total cost

这等价于:

maximize performance

6 MDP —— Markov Decision Process

这一部分是后续大题核心，必须理解。

MDP = Markov Decision Process (马尔可夫决策过程)

用于:

- 在随机环境中做决策

- 状态转移带概率

MDP 定义为五元组:

$$(S, A, P, R, \gamma)$$

解释:

- S : 状态集合 (所有可能状态)
- A : 动作集合
- $P(s'|s, a)$: 从 s 执行动作 a 到达 s' 的概率
- $R(s, a)$: 奖励函数
- γ : 折扣因子 ($0 \leq \gamma < 1$)

为什么叫 Markov?

因为满足:

$$P(s_{t+1}|s_t, s_{t-1}, \dots) = P(s_{t+1}|s_t)$$

未来只依赖当前状态。

7 Bellman Equation (贝尔曼方程)

价值函数:

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$$

解释:

当前价值 = 当前奖励 + 折扣后的未来最优价值

考试如何考?

给:

- 状态
- 转移概率
- 奖励
- γ

要求:

算一轮 value iteration。

步骤必须写:

1. 对每个 action 计算期望
2. 取最大

8 Q-learning (强化学习)

更新公式:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

解释:

- α : 学习率
- R : 即时奖励
- γ : 折扣

考试可能:

给数值, 让你算一次更新。

9 CSP (约束满足问题)

CSP = Constraint Satisfaction Problem

组成:

- Variables
- Domains
- Constraints

样题已经考:

- Forward Checking
- Constraint Propagation

Forward checking:

只检查已赋值变量的邻居。

Constraint propagation:

若某变量 domain 改变,

继续检查所有相关约束

10 Lecture 1 核心逻辑

整门课逻辑:

Perception $\rightarrow$ Model $\rightarrow$ Decision $\rightarrow$ Action

Search: 确定性决策

MDP: 随机决策

RL: 未知模型下学习决策

11 考试真正核心

已通过样题验证:

- UCS 计算路径
- Forward checking
- Constraint propagation

高概率未来计算题:

- Bellman 更新
- Value iteration
- Q-learning 单步更新

所有算法都在实现 Rational Decision-Making

Lecture 2: Linear Regression and Classification

12 Machine Learning Overview

基本模型:

$$x \rightarrow f \rightarrow y$$

- x : 输入 (features)
- f : 模型 (predictor)
- y : 输出 (label 或实数)

三类任务:

- Binary classification: $y \in \{+1, -1\}$
- Regression: $y \in \mathbb{R}$
- Structured prediction: y 为复杂结构

13 Linear Regression (线性回归)

1. Hypothesis Class

线性函数:

$$f_w(x) = w_1 + w_2x$$

向量形式:

$$\phi(x) = [1, x], \quad w = [w_1, w_2], \quad f_w(x) = w \cdot \phi(x)$$

2. Loss Function

平方损失:

$$Loss(x, y, w) = (f_w(x) - y)^2$$

训练损失:

$$TrainLoss(w) = \frac{1}{|D_{train}|} \sum_{(x,y) \in D_{train}} (f_w(x) - y)^2$$

3. Optimization

目标:

$$\min_w \text{TrainLoss}(w)$$

梯度:

$$\nabla_w \text{TrainLoss}(w) = \frac{1}{|D|} \sum 2(f_w(x) - y)\phi(x)$$

梯度下降更新:

$$w \leftarrow w - \eta \nabla_w \text{TrainLoss}(w)$$

其中 η 为 learning rate。

14 Linear Classification (线性分类)

1. Hypothesis Class

预测函数:

$$f_w(x) = \text{sign}(w \cdot \phi(x))$$

决策边界:

$$w \cdot \phi(x) = 0$$

2. Score 与 Margin

Score:

$$\text{score} = w \cdot \phi(x)$$

Margin:

$$\text{margin} = (w \cdot \phi(x))y$$

若 $\text{margin} > 0$, 预测正确。

3. Hinge Loss

$$Loss_{hinge}(x, y, w) = \max\{1 - (w \cdot \phi(x))y, 0\}$$

梯度:

$$\nabla_w = \begin{cases} -\phi(x)y & \text{if } 1 - (w \cdot \phi(x))y > 0 \\ 0 & \text{otherwise} \end{cases}$$

4. Logistic Loss

$$Loss_{logistic}(x, y, w) = \log(1 + e^{-(w \cdot \phi(x))y})$$

15 Regression vs Classification 对比

	Regression	Classification
输出	$\mathbb{R}$	± 1
误差	residual	margin
损失	squared	hinge/logistic
优化	gradient descent	gradient descent

Lecture 2 = 计算梯度 + 参数更新

Lecture 3: Linear Classifier for Image Classification

本讲核心问题：如何用一个线性模型对图像进行多类别分类？

16 Image Classification Problem (图像分类问题)

输入：一张图像，展平为向量 $x \in \mathbb{R}^d$ (例如 $32 \times 32 \times 3 = 3072$ 维)。

输出：类别标签 $y \in \{1, 2, \dots, K\}$ 。

核心挑战 (Challenges):

- Viewpoint variation (视角变化)
- Illumination (光照变化)
- Deformation (形变)
- Occlusion (遮挡)
- Intraclass variation (类内差异)

概念题可能考：为什么图像分类困难？列举 challenges。

17 Nearest Neighbor (最近邻)

基本思想：找训练集中与测试图像最相似的样本，用其标签作为预测。

L1 距离 (Manhattan Distance):

$$D_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

L2 距离 (Euclidean Distance):

$$D_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

K-NN (K 近邻): 找 K 个最近邻，取多数投票：

$$f(x) = \text{majority label among K nearest neighbors}$$

K 的选择：通过 validation set 调参。K 太小 $\rightarrow$ 过拟合；K 太大 $\rightarrow$ 欠拟合。

KNN 在图像分类中实际效果差：测试时间 $O(N)$ ，且像素距离不反映语义相似度。

18 Multi-class Linear Classifier (多类线性分类器)

核心公式:

$$f(x) = Wx + b$$

其中 $W \in \mathbb{R}^{K \times d}$, $b \in \mathbb{R}^K$ 。

输出为每个类别的得分向量 $f \in \mathbb{R}^K$, 预测:

$$\hat{y} = \arg \max_k f_k(x)$$

直觉理解: W 的第 k 行相当于第 k 类的“模板” (template), 得分 = 模板与输入的内积 (相似度)。

逐符号解释:

- W 的第 k 行 $w_k \in \mathbb{R}^d$: 第 k 类的权重向量
- $f_k(x) = w_k \cdot x + b_k$: 第 k 类的得分
- b_k : 偏置 (bias), 调整各类基准分

19 Non-linear Features (非线性特征映射)

线性分类器的决策边界是超平面, 但很多问题线性不可分。

解决方案: 不改变分类器, 改变特征 $\phi(x)$ 。

示例: 原始特征 $x \in \mathbb{R}$, 映射为 $\phi(x) = [1, x, x^2]$ 。

在原始空间非线性 $\rightarrow$ 在特征空间 $\phi(x)$ 仍是线性决策边界。

概念题: “线性分类器如何处理非线性可分数据?”

答: 通过非线性特征映射 $\phi(x)$, 使数据在新特征空间中线性可分。

20 KNN vs Linear Classifier 对比

	KNN	Linear Classifier
类型	Non-parametric (非参数)	Parametric (参数化)
训练时间	$O(1)$ (只存数据)	需要优化 (梯度下降)
测试时间	慢 $O(Nd)$	快 $O(dK)$
存储	需保存全部训练数据	只需保存 W, b
决策边界	复杂 (Voronoi)	线性超平面

21 Hyperparameter 选择与 Cross-Validation

数据划分：

- Training set：训练模型参数
- Validation set：调超参数（如 K、学习率）
- Test set：最终评估，**只能使用一次**

K-Fold Cross-Validation：将训练数据分为 K 份，轮流用其中 1 份做验证，其余做训练，取平均性能。

关键原则：Test set 绝不能用于任何决策（选模型、调参数）。

Lecture 3 = 图像表示 + 多分类线性模型 + 特征非线性化

Lecture 4: Neural Networks and Backpropagation

本讲核心问题：如何自动学习非线性特征？如何高效计算多层网络的梯度？

22 从线性到非线性：为什么需要神经网络

线性分类器的局限：决策边界只能是超平面，无法处理 XOR 等非线性问题。

解决思路：堆叠多个线性变换 + 非线性激活函数 = 万能函数逼近器。

23 Neural Network Structure (神经网络结构)

单层（全连接层）：

$$h = \sigma(Wx + b)$$

多层网络（一般形式）：

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)})$$

逐符号解释：

- $h^{(l)}$ ：第 l 层的激活值（hidden representation）
- $W^{(l)}$ ：第 l 层的权重矩阵
- $\sigma(\cdot)$ ：非线性激活函数
- $h^{(0)} = x$ ：网络输入

24 Activation Functions (激活函数)

名称	公式	优点	缺点
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	输出 (0, 1)	梯度消失、非零中心
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	零中心	仍有梯度消失
ReLU	$\max(0, x)$	计算快、不饱和	Dead neuron ($x < 0$ 梯度为 0)

概念题：“为什么 ReLU 比 Sigmoid 更常用？”

答：(1) 计算简单；(2) 正区间梯度恒为 1，缓解梯度消失；(3) 产生稀疏激活。

25 Backpropagation (反向传播)

核心思想：利用链式法则 (Chain Rule)，从输出层向输入层逐层传递梯度。

链式法则：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x}$$

直觉：每个节点的梯度 = 上游梯度 (upstream gradient) $\times$ 局部梯度 (local gradient)。

26 Computational Graph 中的梯度传播规则

加法门 (Add Gate): $q = x + y$

梯度直接传递: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial q}$, $\frac{\partial L}{\partial y} = \frac{\partial L}{\partial q}$

乘法门 (Multiply Gate): $q = x \cdot y$

交叉传递: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial q} \cdot y$, $\frac{\partial L}{\partial y} = \frac{\partial L}{\partial q} \cdot x$

Max 门 (Max Gate): $q = \max(x, y)$

梯度路由到较大者 (router)，另一路梯度为 0。

Sigmoid 导数：

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

计算题高频考点：给一个小的 computational graph，要求算出各节点的梯度。

步骤：(1) 前向计算各节点值；(2) 从 loss 开始反向，逐节点用链式法则。

27 2-Layer Network 训练示例

前向传播：

$$h = \sigma(XW_1), \quad \hat{y} = hW_2$$

反向传播求梯度：

$$\nabla_{W_2} = h^T(\hat{y} - y)$$

$$\nabla_{W_1} = X^T [(\hat{y} - y)W_2^T \odot \sigma'(XW_1)]$$

其中 $\odot$ 为逐元素乘 (Hadamard product), $\sigma'(s) = h \odot (1 - h)$ (Sigmoid 情况下)。

更新：

$$W \leftarrow W - \eta \nabla_W L$$

28 Vanishing Gradient Problem (梯度消失)

问题：Sigmoid/Tanh 在饱和区导数接近 0，经过多层连乘后梯度趋近于 0，底层参数无法更新。

解决方案：

- 使用 ReLU 激活函数
- Residual connections (ResNet)
- Batch Normalization
- 合理的权重初始化 (如 Xavier/He initialization)

$\text{Backprop} = \text{上游梯度} \times \text{局部梯度}, \text{逐层反向传递}$

Lecture 5: Unsupervised Learning and Generalization

本讲两大主题：(1) 无监督聚类 (K-means)；(2) 训练技巧与泛化能力。

29 K-means Clustering (K 均值聚类)

目标：将 n 个数据点分为 K 组，使每个点到其所属聚类中心的距离之和最小。

目标函数：

$$\text{Loss}_{\text{kmeans}}(z, \mu) = \sum_{i=1}^n \|x_i - \mu_{z_i}\|^2$$

逐符号解释：

- x_i ：第 i 个数据点
- $z_i \in \{1, \dots, K\}$ ： x_i 的簇分配
- μ_k ：第 k 个聚类中心 (centroid)

交替优化算法 (Alternating Optimization)

Step 1 — 固定 μ ，更新 z (分配步)：

$$z_i = \arg \min_k \|x_i - \mu_k\|^2$$

即：把每个点分配到最近的中心。

Step 2 — 固定 z ，更新 μ (更新步)：

$$\mu_k = \frac{1}{|\{i : z_i = k\}|} \sum_{i: z_i = k} x_i$$

即：每个中心移动到其成员的均值位置。

收敛性：每步都不增加 Loss，一定收敛——但只保证到**局部最优**，不保证全局最优。

初始化影响：不同初始中心可能导致不同结果。实践中常用 K-means++ 或多次随机重启取最优。

概念题：“K-means 是否保证找到全局最优？”

答：不保证。K-means 只收敛到局部最优，结果依赖初始化。

30 SGD (随机梯度下降)

Full Gradient Descent: 每步用全部数据计算梯度, 计算量大。

Stochastic Gradient Descent (SGD): 每步只用一个 (或一小批) 样本:

$$w \leftarrow w - \eta \nabla_w \text{Loss}(x_i, y_i, w)$$

优点: 计算快、可处理大数据集、噪声有正则化效果。

Mini-batch SGD: 折中方案, 每步用 B 个样本的平均梯度。

31 Overfitting 与 Generalization (过拟合与泛化)

核心目标: 最小化 test error (泛化误差), 而非 train error。

过拟合 (Overfitting): 模型在训练集表现好, 测试集差。原因: 模型过于复杂, 记住了噪声。

欠拟合 (Underfitting): 训练集和测试集都差。原因: 模型过于简单。

正则化 (Regularization): 限制模型复杂度:

$$\min_w \text{TrainLoss}(w) + \frac{\lambda}{2} \|w\|^2$$

逐符号解释:

- λ : 正则化强度。 λ 越大, 越倾向于小的权重 (简单模型)。
- $\|w\|^2 = \sum_j w_j^2$: L2 正则 (Weight Decay)

其他防止过拟合的方法:

- Early stopping (早停)
- Dropout (随机丢弃神经元)
- Data augmentation (数据增强)
- 减小模型容量

关键原则: 测试集绝对不能用于调参或模型选择。

真实目标 = 泛化误差 (test error)

Lecture 6: Uninformed Search

本讲核心问题：在确定性环境中，如何系统地搜索从起点到目标的路径？

32 Search Problem 定义

搜索问题由四部分组成：

- **State space** (状态空间)：所有可能的状态
- **Successor function** (后继函数)： $Successors(s) = \{(s', action, cost)\}$
- **Start state** (初始状态)
- **Goal test** (目标测试)：判断当前状态是否为目标

解 (Solution)：从起始状态到目标状态的一条动作序列。

最优解 (Optimal Solution)：总代价最小的解。

33 Tree Search vs Graph Search

Tree Search：可以重复访问同一状态（可能无限循环）。

Graph Search：维护 explored set（已访问集合），避免重复展开同一状态。

关键区别：Graph Search 需要额外空间记录已访问节点，但避免了冗余展开。考试中如果题目未特别说明，通常按 Graph Search 做。

34 搜索算法详解

DFS (深度优先搜索)

数据结构：Stack (后进先出)

策略：总是展开最深的节点。

性质：

- 时间复杂度： $O(b^m)$ (b = branching factor, m = 最大深度)
- 空间复杂度： $O(bm)$ (只存储一条路径 + 兄弟节点)
- Complete (完备性)：在有限状态空间中是完备的 (graph search)；tree search 中可能不完备 (无限循环)
- Optimal：不是最优的

BFS (广度优先搜索)

数据结构: Queue (先进先出)

策略: 总是展开最浅的节点。

性质:

- 时间复杂度: $O(b^s)$ (s = 最浅解的深度)
- 空间复杂度: $O(b^s)$
- Complete: 是 (若 b 有限)
- Optimal: 仅当所有边代价相等时最优

IDS (迭代加深搜索)

思想: 结合 DFS 的低空间和 BFS 的最优性。依次做深度限制为 1, 2, 3, ... 的 DFS。

性质:

- 时间: $O(b^s)$ (与 BFS 同阶, 重复展开的代价被 $b/(b-1)$ 的常数因子消化)
- 空间: $O(bs)$ (与 DFS 同阶)
- Complete & Optimal (单位代价时)

UCS (一致代价搜索, Uniform Cost Search)

数据结构: Priority Queue (优先队列, 按 $g(n)$ 排序)

策略: 总是展开代价最小的节点。

展开规则:

Expand node n with smallest $g(n)$ = 从起点到 n 的路径代价

性质:

- 时间/空间复杂度: $O(b^{\lceil C^*/\epsilon \rceil})$ (C^* = 最优解代价, ϵ = 最小边代价)
- Complete: 是 (只要边代价 > 0)
- Optimal: 是 (关键性质!)

为什么 UCS 是最优的?

UCS 按代价从小到大展开节点, 第一次展开目标节点时, 其路径代价一定是所有到达目标路径中最小的。

UCS 是样卷考点！需要能手动模拟 UCS 展开过程：

- (1) 初始：frontier = {start}；
- (2) 每步：弹出代价最小的节点，展开其所有后继；
- (3) 若后继已在 frontier 且新代价更低，则更新；
- (4) 当目标节点被弹出（不是被加入）时停止。

35 算法对比总结

Algorithm	数据结构	Time	Space	Optimal?
DFS	Stack	$O(b^m)$	$O(bm)$	No
BFS	Queue	$O(b^s)$	$O(b^s)$	Yes (unit cost)
IDS	Stack	$O(b^s)$	$O(bs)$	Yes (unit cost)
UCS	Priority Queue	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$	Yes

36 UCS 手动展开示例

题设：图中 $S \rightarrow A$ (cost 1), $S \rightarrow B$ (cost 4), $A \rightarrow B$ (cost 2), $A \rightarrow G$ (cost 5), $B \rightarrow G$ (cost 2)。

展开过程：

1. Frontier: $\{S : 0\}$ 。弹出 S ，展开得 $A : 1, B : 4$ 。
2. Frontier: $\{A : 1, B : 4\}$ 。弹出 $A : 1$ ，展开得 $B : 3, G : 6$ 。更新 B 为 3。
3. Frontier: $\{B : 3, G : 6\}$ 。弹出 $B : 3$ ，展开得 $G : 5$ 。更新 G 为 5。
4. Frontier: $\{G : 5\}$ 。弹出 $G : 5$ ，目标！停止。

最优路径： $S \rightarrow A \rightarrow B \rightarrow G$ ，代价 = 5。

UCS = 优先队列 + 按 $g(n)$ 展开 = 最优

Lecture 7: Informed Search

本讲核心问题：如何利用”对目标的估计”来加速搜索？

37 Informed Search 的动机

UCS 盲目地按代价展开，可能探索大量远离目标的节点。

改进思路：引入 heuristic function $h(n)$ ，估计从 n 到目标的代价，引导搜索方向。

38 Greedy Best-First Search（贪心最佳优先搜索）

策略：展开 $h(n)$ 最小的节点（看起来离目标最近的）。

性质：

- 速度快（如果 h 好的话）
- **不保证最优**（可能被误导到长路径）
- 不一定完备

39 A* Search

核心公式：

$$f(n) = g(n) + h(n)$$

逐符号解释：

- $g(n)$ ：从起点到 n 的**实际代价**（backward cost）
- $h(n)$ ：从 n 到目标的**估计代价**（forward cost）
- $f(n)$ ：经过 n 的总估计代价

策略：展开 $f(n) = g(n) + h(n)$ 最小的节点。

直觉： $A^* = \text{UCS}$ （保证最优的后向代价）+ Greedy（利用前向估计加速）。

40 Admissibility（可容许性）

定义：

$$h(n) \leq h^*(n) \quad \forall n$$

其中 $h^*(n)$ 是从 n 到目标的真实最优代价。

含义： Admissible heuristic 永远不会高估到目标的代价（乐观估计）。

定理： 若 h 是 admissible 的，A* Tree Search 是最优的。

41 Consistency (一致性 / 单调性)

定义： 对任何节点 n 及其后继 n' ：

$$h(n) \leq cost(n, n') + h(n')$$

这是三角不等式的一种形式。

性质：

- Consistency $\Rightarrow$ Admissibility (一致性蕴含可容许性)
- 反之不一定成立
- A* Graph Search 需要 consistency 来保证最优性

考试常见问法：

(1) “判断以下 heuristic 是否 admissible”——只需检查是否 $\leq h^*$ 。

(2) “A* 在什么条件下是 optimal 的？”——Tree search: admissible; Graph search: consistent。

42 Heuristic 设计方法

Relaxed Problem (松弛问题)： 去掉原问题的某些约束，得到更容易解的问题。
 松弛问题的最优解 = 原问题的一个 admissible heuristic。

示例 (8-puzzle)：

- h_1 = 错位 tile 数量 (Misplaced Tiles)：松弛 “tile 可以直接跳到目标位置”
- h_2 = Manhattan Distance：松弛 “tile 可以穿过其他 tile”

43 Heuristic 的组合与比较

Dominance (支配)： 若 $h_1(n) \geq h_2(n)$ 对所有 n 成立，则 h_1 支配 h_2 。 h_1 更好 (展开更少节点)。

组合： 若 h_1, h_2 都是 admissible 的，则：

$$h(n) = \max\{h_1(n), h_2(n)\}$$

也是 admissible 的, 且支配 h_1 和 h_2 。

概念题: “如何构造更好的 admissible heuristic? ”

答: (1) 用松弛问题; (2) 取多个 admissible heuristic 的 max。

$$A^* = \text{UCS} + \text{Heuristic}, \text{ admissible} \Rightarrow \text{optimal}$$

Lecture 8: Constraint Satisfaction Problems

本讲核心问题：如何在满足一组约束的前提下，为变量找到合法赋值？

44 CSP 定义

CSP = Constraint Satisfaction Problem (约束满足问题)

三要素：

- **Variables** (变量): $X_1, X_2, \dots, X_n$
- **Domains** (域): 每个变量的取值范围 D_i
- **Constraints** (约束): 变量间必须满足的关系

经典例子：Map Coloring (地图着色)、N-Queens、数独。

45 Backtracking Search (回溯搜索)

基本思想：DFS + 一次只赋值一个变量 + 约束检查。

关键优化：

- Branching factor = d (domain size), 不是 nd
- 若发现当前赋值违反约束，立即回溯

46 Variable Ordering: MRV

MRV (Minimum Remaining Values): 选择剩余合法值最少的变量优先赋值。

直觉：“Fail-first”策略——最受约束的变量最容易导致失败，尽早尝试可以减少搜索树大小。

47 Value Ordering: LCV

LCV (Least Constraining Value): 选择对邻居约束影响最小的值优先尝试。

直觉：“Succeed-first”——选择保留最多灵活性的值，增大后续赋值成功的概率。

注意区分：MRV 选变量 (选最受约束的), LCV 选值 (选约束最少的)。策略相反！

48 Forward Checking (前向检查)

规则：每当给变量 X_i 赋值后，检查所有与 X_i 有约束关系的未赋值邻居，从邻居的 domain 中删除与当前赋值不一致的值。

特点：

- 只看一步（只检查直接邻居）
- 若某邻居 domain 变为空集 $\rightarrow$ 当前赋值失败，回溯
- 不传播（不会因为邻居 domain 变化而继续检查邻居的邻居）

示例：

假设 $X_1 \in \{R, G, B\}$, $X_2 \in \{R, G, B\}$, $X_3 \in \{R, G, B\}$, 约束 $X_1 \neq X_2$, $X_2 \neq X_3$ 。

赋值 $X_1 = R$: Forward checking 删除 X_2 中的 $R \rightarrow X_2 \in \{G, B\}$ 。 X_3 不是 X_1 的邻居（无直接约束），不变。

49 Arc Consistency (弧一致性)

定义：弧 (X_i, X_j) 是一致的 (arc-consistent)，如果对 X_i domain 中的每一个值， X_j 的 domain 中都存在至少一个与之兼容的值。

形式化：

$$\forall v \in D_i, \exists w \in D_j \text{ s.t. } (v, w) \text{ satisfies constraint}(X_i, X_j)$$

AC-3 算法步骤：

1. 将所有弧加入队列 Q 。
2. 从 Q 取出弧 (X_i, X_j) ，删除 D_i 中所有没有 D_j 支持的值。
3. 若 D_i 发生变化，将所有 (X_k, X_i) ($k \neq j$) 加入 Q 。
4. 若某 domain 变空 $\rightarrow$ 无解。
5. 重复直到 Q 为空。

样卷考点！Forward Checking vs Arc Consistency 的区别：

FC 只看赋值变量的直接邻居，不传播。

AC 会持续传播：若某变量 domain 改变，重新检查相关弧，可能级联删除。

50 Forward Checking vs Arc Consistency 对比

	Forward Checking	Arc Consistency (AC-3)
触发条件	赋值后	任何 domain 变化后
检查范围	只看直接邻居	传播到所有相关弧
是否传播	不传播	传播
检测能力	较弱	较强（能更早发现矛盾）
开销	较小	较大 $O(ed^3)$

51 Min-Conflicts（最小冲突，局部搜索）

思想： 从一个完整赋值开始，迭代修改冲突变量的值。

步骤：

1. 随机选一个有冲突的变量。
2. 将其赋值改为使冲突数最小的值。
3. 重复直到无冲突或达到最大迭代次数。

优点： 对大规模 CSP（如百万皇后）效果好。

CSP = Backtracking + Constraint Propagation (FC / AC)

Lecture 9: Local Search Algorithms

本讲核心问题：当状态空间太大无法系统搜索时，如何用局部搜索找到”足够好”的解？

52 Local Search 的特点

- 不维护搜索树，只保持当前状态
- 不关心路径，只关心最终解的质量
- 适用于**优化问题**（找最大/最小值）
- 内存开销极小： $O(1)$

53 Hill Climbing (爬山法)

策略：总是移动到当前邻居中最好的状态。若没有更好的邻居则停止。

问题：

- **Local optima** (局部最优)：邻居都不如当前状态，但不是全局最优
- **Plateaus** (平台)：邻居值相同，无法判断方向
- **Ridges** (山脊)：无法一步到达更高点

改进方法：

- Random restart (随机重启)：多次从随机初始点开始
- Stochastic hill climbing：随机选择一个改进邻居（非最佳）

54 Simulated Annealing (模拟退火)

核心思想：允许以一定概率接受”变差”的移动，从而跳出局部最优。

接受概率：

$$P(\text{accept worse move}) = e^{\Delta E/T}$$

逐符号解释：

- $\Delta E = f(\text{new}) - f(\text{current})$ ：若最大化问题， $\Delta E < 0$ 表示变差
- T ：温度参数

- T 高 $\rightarrow$ 接受差解概率高 (exploration 探索)
- $T \rightarrow 0 \rightarrow$ 退化为 Hill Climbing (exploitation 利用)

Temperature Schedule (退火计划): T 随时间逐渐降低。

理论保证: 若降温足够慢, 模拟退火以概率 1 收敛到全局最优 (实际中不可行, 太慢)。

概念题: “Simulated Annealing 如何避免局部最优?”

答: 通过以概率 $e^{\Delta E/T}$ 接受更差的状态, 温度高时探索力强。随温度降低逐渐收敛。

55 Genetic Algorithm (遗传算法)

思想: 模拟自然选择, 维护一个种群, 通过选择、交叉、变异迭代进化。

流程:

1. **初始化:** 随机生成种群
2. **评估:** 计算每个个体的 Fitness
3. **选择 (Selection):** 按 fitness 比例选择父代
4. **交叉 (Crossover):** 两父代交换部分基因产生后代
5. **变异 (Mutation):** 随机改变后代的某些基因
6. 重复 2-5

Selection probability (选择概率):

$$Pr(i) = \frac{Fitness(i)}{\sum_j Fitness(j)}$$

Crossover 示例: 单点交叉——选一个切割点, 两父代互换后半部分。

Mutation 作用: 维持种群多样性, 防止过早收敛。

Local Search: Hill Climbing (贪心)、SA (随机接受)、GA (种群进化)

Lecture 10: Markov Decision Process I

本讲核心问题：在随机环境中如何做序贯决策？如何形式化“随机搜索”？

56 从 Search 到 MDP

Search (Lecture 6–7) 假设确定性环境：一个 action 只有一个 successor。

MDP 的区别：一个 action 可能产生多个 successor，每个带概率。

关键对比：

	Search Problem	MDP
环境	Deterministic	Stochastic
目标	找最优路径	找最优 policy
解	Action 序列	$\pi : S \rightarrow A$ (状态到动作的映射)

57 MDP 定义

六元组： $(States, s_{start}, Actions, T, Reward, IsEnd)$

- $States$: 状态空间
- s_{start} : 起始状态
- $Actions(s)$: 状态 s 下可执行的动作集合
- $T(s, a, s')$: 转移概率函数 (Transition Function), $P(s'|s, a)$
- $Reward(s, a, s')$: 执行 a 从 s 转移到 s' 获得的即时奖励
- $IsEnd(s)$: 判断是否为终端状态

58 Discount Factor (折扣因子)

为什么需要 γ ?

- 未来奖励不如当前奖励有价值
- 避免无限 horizon 下总收益发散
- $\gamma \in [0, 1)$: γ 越小越短视, γ 越大越远视

折扣回报 (Discounted Utility):

$$U = R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots = \sum_{t=0}^{\infty} \gamma^t R_t$$

有限界: $U \leq \frac{R_{max}}{1-\gamma}$

59 Policy (策略)

定义： $\pi : States \rightarrow Actions$ ，指定在每个状态下应该执行什么动作。

MDP 的目标： 找到使期望折扣回报最大的最优策略 π^* 。

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid \pi \right]$$

60 Value Function (价值函数)

状态价值函数 $V_{\pi}(s)$ ： 从状态 s 出发，按策略 π 行动，获得的期望折扣回报。

Policy Evaluation (策略评估) —— Bellman 期望方程：

$$V_{\pi}(s) = \sum_{s'} T(s, \pi(s), s') [Reward(s, \pi(s), s') + \gamma V_{\pi}(s')]$$

直觉： 当前状态价值 = 即时奖励 + 折扣后的下一状态价值的期望。

61 Bellman Optimality Equation (贝尔曼最优方程)

$$V_{opt}(s) = \max_{a \in Actions(s)} \sum_{s'} T(s, a, s') [Reward(s, a, s') + \gamma V_{opt}(s')]$$

与 Policy Evaluation 的区别： 多了 $\max_a$ ，表示选择最优动作。

62 Value Iteration (值迭代)

算法：

初始化 $V^{(0)}(s) = 0$ (所有状态)。迭代：

$$V^{(t)}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{(t-1)}(s')]$$

收敛条件： 当 $\max_s |V^{(t)}(s) - V^{(t-1)}(s)| < \epsilon$ 时停止。

策略提取：

$$\pi_{opt}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

这是大题核心！考试会给具体数值让你手算 1-2 轮 Value Iteration。

务必注意：(1) 终端状态 $V = 0$ 不更新；(2) γ 乘 V 不乘 R ；(3) 每步对所有动作算 Q 取 $\max$ 。

$$V^{(t)}(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} T(s, a, s') V^{(t-1)}(s') \right]$$

Lecture 11: Markov Decision Process II

本讲核心问题：除了 Value Iteration，还有什么更高效的方法求解 MDP？

63 Policy Iteration (策略迭代)

思想：交替进行”策略评估”和”策略改进”，直到策略不再变化。

Step 1 —Policy Evaluation (策略评估)

给定固定策略 π ，求解 $V_\pi(s)$ 。

$$V_\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_\pi(s')]$$

这是一个线性方程组 ($|S|$ 个方程, $|S|$ 个未知数), 可直接求解或迭代求解。

与 Value Iteration 的区别：这里没有 $\max_a$ ——动作由 π 固定。

Step 2 —Policy Improvement (策略改进)

用当前 V_π 贪心地更新策略：

$$\pi_{new}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_\pi(s')]$$

若 $\pi_{new} = \pi$ ，则已找到最优策略，停止。

Policy Iteration 完整流程

1. 初始化任意策略 π_0
2. **Evaluate:** 计算 V_{π_k}
3. **Improve:** $\pi_{k+1}(s) = \arg \max_a Q_{\pi_k}(s, a)$
4. 若 $\pi_{k+1} = \pi_k$ ，停止；否则回到 Step 2

64 Value Iteration vs Policy Iteration

	Value Iteration	Policy Iteration
每步操作	更新 $V(s)$ (所有状态)	完整评估 π + 改进 π
收敛性	值逐渐收敛	策略有限步内收敛
每步代价	$O(S ^2 A)$	评估 $O(S ^3)$ + 改进 $O(S ^2 A)$
迭代次数	可能较多	通常很少 (策略数有限)
实践表现	状态空间大时慢	通常更快收敛

概念题：“VI 和 PI 的主要区别是什么？”

答：VI 直接迭代更新值函数（每步取 $\max$ ）；PI 交替做完整策略评估（解方程组）和策略改进。PI 通常迭代次数更少。

65 Q-Value 函数

定义：

$$Q_{opt}(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_{opt}(s')]$$

与 V 的关系：

$$V_{opt}(s) = \max_a Q_{opt}(s, a)$$

策略提取：

$$\pi^*(s) = \arg \max_a Q_{opt}(s, a)$$

为什么 Q 有用？知道 Q 就能直接得到最优策略，不需要知道 T 和 R 。这是 Q-learning 的基础。

PI = Evaluate (解方程组) + Improve (贪心改进), 通常更快

Lecture 12: Reinforcement Learning I

本讲核心问题：当转移概率 T 和奖励 R 未知时，如何通过与环境交互来学习最优策略？

66 RL vs Planning 的区别

	Planning (MDP)	Reinforcement Learning
$T(s, a, s')$	已知	未知
$R(s, a, s')$	已知	未知（只能观察样本）
方法	Value Iteration / Policy Iteration	与环境交互，从经验中学习

67 Model-Based RL (基于模型的 RL)

思想：先从经验中估计 T 和 R ，再用 Planning 方法（如 Value Iteration）求解。

估计转移概率：

$$\hat{T}(s, a, s') = \frac{\#(s, a, s')}{\#(s, a)}$$

估计奖励：

$$\hat{R}(s, a, s') = \text{观察到的}(s, a, s') \text{ 转移的平均奖励}$$

优点：数据利用效率高。

缺点：需要大量数据才能准确估计 T ；模型偏差可能传播。

68 Model-Free RL (无模型 RL)

不显式学习 T 和 R ，直接学习值函数或策略。

Direct Evaluation (直接评估)

思想：执行策略 π ，记录从每个状态出发获得的实际回报，取平均。

步骤：

1. 按策略 π 生成多条 episode
2. 对每个状态 s ，记录所有从 s 开始的折扣回报
3. $V_\pi(s) \approx$ 这些回报的平均值

优点：简单、无偏。

缺点：收敛慢；没有利用 Bellman 结构（状态间的关系）；每个状态独立估计。

69 Temporal Difference Learning (TD 学习)

动机：利用 Bellman 方程的结构，无需等到 episode 结束就能更新。

TD 更新规则：

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha [r + \gamma V^\pi(s') - V^\pi(s)]$$

逐符号解释：

- α : 学习率 (step size)
- r : 实际获得的即时奖励
- s' : 实际转移到的下一状态
- $r + \gamma V^\pi(s')$: TD target (用一步经验 + 当前估计来逼近真实值)
- $r + \gamma V^\pi(s') - V^\pi(s)$: TD error (δ)

直觉：朝着“一步经验 + 当前估计”的方向更新，bootstrapping (自举)。

TD vs Direct Evaluation：

	Direct Evaluation	TD Learning
更新时机	Episode 结束后	每步都能更新
利用结构	否	是 (Bellman 结构)
收敛速度	慢	快
偏差	无偏	有偏 (bootstrap)

考试可能要求：给定一个 (s, a, r, s') 序列，用 TD 规则更新 $V(s)$ 。

TD Learning: $V(s) \leftarrow V(s) + \alpha \underbrace{[r + \gamma V(s')] - V(s)}_{\text{TD target}}$
--

Lecture 13: Reinforcement Learning II

本讲核心问题：如何在不知道模型的情况下学习**最优策略**（而非评估给定策略）？

70 Q-Learning

动机： TD Learning 评估给定策略 π 的 V_π 。但我们想直接学习最优 Q^* ，无需知道 T 。

Q-Learning 更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

逐符号解释：

- $Q(s, a)$: 状态 s 执行动作 a 的当前估计值
- r : 执行 a 后获得的即时奖励
- s' : 转移到的下一状态
- $\max_{a'} Q(s', a')$: 下一状态的最优估计值
- α : 学习率
- 方括号内 = TD error

关键特性—Off-policy（离策略）：

- 行为策略 (behavior policy): 用来探索的策略 (如 ϵ -greedy)
- 目标策略 (target policy): 要学习的最优策略 (greedy on Q)
- Q-learning 可以用任意探索策略收集数据，同时学习最优策略

71 Exploration vs Exploitation

矛盾： 利用已知最优动作 (exploitation) vs 尝试新动作获取信息 (exploration)。

ϵ -Greedy 策略：

$$a = \begin{cases} \arg \max_a Q(s, a) & \text{with probability } 1 - \epsilon \\ \text{random action} & \text{with probability } \epsilon \end{cases}$$

ϵ 通常随时间衰减：初期多探索，后期多利用。

72 SARSA (On-policy TD Control)

更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

与 Q-learning 的区别：

	Q-Learning	SARSA
类型	Off-policy	On-policy
Target	$r + \gamma \max_{a'} Q(s', a')$	$r + \gamma Q(s', a')$ (实际执行的 a')
学习的策略	最优策略	当前行为策略
安全性	可能学到”大胆”策略	更保守 (考虑实际探索风险)

73 Approximate Q-Learning (函数逼近 Q 学习)

问题：当状态空间巨大时，无法为每个 (s, a) 维护一个 Q 值。

解决方案：用特征的线性组合近似 Q 函数：

$$Q(s, a) = \sum_i w_i f_i(s, a) = \mathbf{w} \cdot \mathbf{f}(s, a)$$

更新权重：

$$w_i \leftarrow w_i + \alpha \cdot \underbrace{\left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]}_{\text{TD error}} \cdot f_i(s, a)$$

直觉：与线性回归类似——用 TD error 作为”目标与预测的差”，乘以特征（梯度方向）来更新权重。

74 Deep Q-Network (DQN)

核心思想：用深度神经网络代替线性函数逼近 $Q(s, a; \theta)$ 。

Loss Function：

$$L(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2$$

两个关键技术：

- **Experience Replay** (经验回放)：将 (s, a, r, s') 存入 buffer，随机采样 mini-batch 训练。打破数据相关性。
- **Target Network** (目标网络)：使用旧参数 θ^- 计算 target，定期更新。稳定训练。

计算题：给定当前 Q 值表和一个 (s, a, r, s') 转移，计算一步 Q-learning 更新后的新 $Q(s, a)$ 。

步骤：(1) 计算 $\text{target} = r + \gamma \max_{a'} Q(s', a')$ ；(2) 计算 $\text{TD error} = \text{target} - Q(s, a)$ ；
(3) 新 $Q(s, a) = Q(s, a) + \alpha \cdot \text{TD error}$ 。

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Lecture 14: Games I

本讲核心问题：在有对手的环境中（对抗搜索），如何做出最优决策？

75 Game 的分类

- **Zero-Sum Game (零和博弈)**：一方的收益 = 另一方的损失。 $U_1(s) = -U_2(s)$ 。
- **Deterministic vs Stochastic**：有无随机因素（如掷骰子）
- **Perfect Information vs Imperfect Information**：能否看到完整游戏状态

本讲聚焦：**确定性、完全信息、零和博弈**（如国际象棋、围棋、井字棋）。

76 Minimax (极小极大)

核心理念：MAX 玩家最大化收益，MIN 玩家最小化 MAX 的收益。双方都做最优决策。

递归定义：

$$V(s) = \begin{cases} Utility(s) & \text{if } s \text{ is terminal} \\ \max_a V(Succ(s, a)) & \text{if MAX's turn} \\ \min_a V(Succ(s, a)) & \text{if MIN's turn} \end{cases}$$

直觉：从叶子节点（终端状态）的 utility 值出发，交替取 max 和 min 回传到根节点。

性质：

- **Complete**：是（有限游戏）
- **Optimal**：是（对抗双方都最优）
- **时间复杂度**： $O(b^m)$ (b = branching factor, m = 最大深度)
- **空间复杂度**： $O(bm)$ (DFS)

问题：对于复杂游戏（国际象棋 $b \approx 35$, $m \approx 100$ ）， b^m 不可计算。

77 Depth-Limited Minimax (有限深度 Minimax)

解决方案： 设定搜索深度上限 d ，在截断深度用 Evaluation Function 估计状态价值。

Evaluation Function (评估函数)：

$$Eval(s) = \sum_i w_i f_i(s)$$

逐符号解释：

- $f_i(s)$ ：特征（如棋子数量、位置优势等）
- w_i ：特征权重（可手工设计或学习得到）

设计原则：

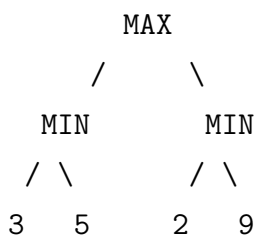
- 终端状态的 Eval 应等于 Utility
- 非终端状态的 Eval 应与获胜概率正相关
- 计算要快

概念题：“为什么需要 Evaluation Function？”

答：因为搜索到底太慢 ($O(b^m)$)，需要在有限深度截断，用 Eval 函数估计叶子节点价值。

78 Minimax 手动计算示例

给定如下 game tree (MAX 先手，深度 2)：



计算过程：

1. 左 MIN 节点： $\min(3, 5) = 3$
2. 右 MIN 节点： $\min(2, 9) = 2$
3. 根 MAX 节点： $\max(3, 2) = 3$

最优策略： MAX 选左子树，获得价值 3。

Minimax: MAX 取 max, MIN 取 min, 交替回传

Lecture 15: Games II —Alpha-Beta Pruning and Expectiminimax

本讲核心问题：如何减少 Minimax 搜索的节点数？如何处理含随机性的博弈？

79 Alpha-Beta Pruning (Alpha-Beta 剪枝)

核心思想：在 Minimax 搜索中，某些分支不可能影响最终结果，可以安全跳过。
维护两个值：

- α : MAX 节点目前能保证的**最低**收益 (MAX 的 best option so far)
- β : MIN 节点目前能保证的**最高**收益 (MIN 的 best option so far)

初始调用： $\alpha = -\infty$, $\beta = +\infty$ 。

剪枝规则

在 MAX 节点：

- 遍历子节点，更新 $\alpha = \max(\alpha, v)$
- 若 $v \geq \beta$ ：剪枝 (MIN 不会允许走到这里)

在 MIN 节点：

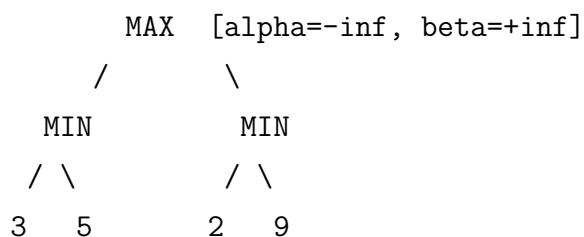
- 遍历子节点，更新 $\beta = \min(\beta, v)$
- 若 $v \leq \alpha$ ：剪枝 (MAX 不会选择这个分支)

记忆口诀：当 $\alpha \geq \beta$ 时剪枝。

性质

- 结果与 Minimax 完全相同 (只是更快)
- 最好情况： $O(b^{m/2})$ (相当于搜索深度翻倍!)
- 最坏情况： $O(b^m)$ (没有剪枝)
- 节点访问顺序影响剪枝效率：最优移动排在前面 $\rightarrow$ 剪枝最多

80 Alpha-Beta 手动计算示例



计算过程：

1. 进入左 MIN 节点 ($\alpha = -\infty, \beta = +\infty$)
2. 左 MIN 看到子节点 3: $\beta = \min(+\infty, 3) = 3$
3. 左 MIN 看到子节点 5: $\beta = \min(3, 5) = 3$ 。返回 3。
4. 回到 MAX: $\alpha = \max(-\infty, 3) = 3$
5. 进入右 MIN 节点 ($\alpha = 3, \beta = +\infty$)
6. 右 MIN 看到子节点 2: $\beta = \min(+\infty, 2) = 2$
7. 此时 $\alpha = 3 \geq \beta = 2$: **剪枝!** 不再看子节点 9。
8. MAX 结果: $\max(3, 2) = 3$

Alpha-Beta 剪枝是高频考点!

关键技能: (1) 标注每个节点的 α, β 值; (2) 判断何时触发剪枝条件 $\alpha \geq \beta$; (3) 标记被剪掉的子树。

81 Expectiminimax (含随机性的博弈)

适用场景: 游戏中有随机事件 (掷骰子、抽牌等)。

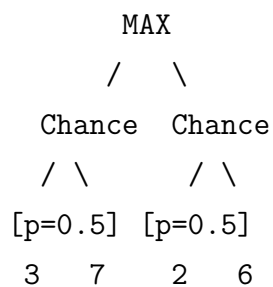
新增 Chance Node (机会节点):

$$V(\text{chance node}) = \sum_{s'} P(s'|s) \cdot V(s')$$

三种节点类型:

- MAX 节点: 取 max
- MIN 节点: 取 min
- Chance 节点: 取加权平均 (期望)

示例:



左 Chance: $0.5 \times 3 + 0.5 \times 7 = 5$

右 Chance: $0.5 \times 2 + 0.5 \times 6 = 4$

MAX: $\max(5, 4) = 5$

重要注意: Alpha-Beta 剪枝**不能直接**应用于 Chance 节点 (因为加权平均不满足 max/min 的剪枝条件)。

Alpha-Beta: $\alpha \geq \beta$ 时剪枝; Expectiminimax 对 Chance 节点取期望

Lecture 16: Probabilistic Reasoning I —Uncertainty

本讲核心问题：如何用概率论处理不确定性？如何从已知概率推导未知概率？

82 为什么需要概率

现实世界中：传感器有噪声、模型不完整、环境随机。

概率 = 对不确定性的量化。

83 核心概率规则

条件概率 (Conditional Probability):

$$P(a | b) = \frac{P(a, b)}{P(b)}$$

含义：在已知 b 发生的前提下， a 发生的概率。

乘法规则 (Product Rule):

$$P(a, b) = P(a | b) \cdot P(b) = P(b | a) \cdot P(a)$$

链式法则 (Chain Rule):

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | x_1, \dots, x_{i-1})$$

边缘化 (Marginalization):

$$P(X) = \sum_y P(X, Y = y)$$

含义：把不关心的变量”求和掉”。

84 Bayes' Rule (贝叶斯规则)

$$P(a | b) = \frac{P(b | a) \cdot P(a)}{P(b)}$$

其中分母可用全概率公式展开：

$$P(b) = \sum_a P(b | a)P(a)$$

直觉：已知”因果方向” $P(b|a)$ ，反推”诊断方向” $P(a|b)$ 。

术语：

- $P(a)$: 先验概率 (Prior)
- $P(b|a)$: 似然 (Likelihood)
- $P(a|b)$: 后验概率 (Posterior)

85 Bayes' Rule 计算示例

题设: 疾病 D 的先验 $P(D) = 0.01$ 。检测灵敏度 $P(+|D) = 0.9$ 。假阳性率 $P(+|\neg D) = 0.05$ 。

求: $P(D|+)$ 。

解:

$$\begin{aligned} P(+) &= P(+|D)P(D) + P(+|\neg D)P(\neg D) \\ &= 0.9 \times 0.01 + 0.05 \times 0.99 = 0.009 + 0.0495 = 0.0585 \\ P(D|+) &= \frac{P(+|D)P(D)}{P(+)} = \frac{0.009}{0.0585} \approx 0.154 \end{aligned}$$

结论: 即使检测阳性, 真正患病的概率只有约 15.4%!

86 Independence 与 Conditional Independence

Independence (独立性):

$$P(A, B) = P(A) \cdot P(B) \quad \Leftrightarrow \quad P(A|B) = P(A)$$

Conditional Independence (条件独立性):

$$P(A, B | C) = P(A | C) \cdot P(B | C)$$

注意: 独立 $\nRightarrow$ 条件独立; 条件独立 $\nRightarrow$ 独立。

概念题: “A 和 B 独立是否意味着给定 C 时它们条件独立?”

答: 不一定。独立与条件独立没有蕴含关系。

87 从 Joint Distribution 提取 Conditional

步骤:

1. Select (选择): 从联合分布表中选出满足条件 (evidence) 的行
2. Normalize (归一化): 使所选行的概率之和为 1

计算题：给一张联合概率表，求某个条件概率。

步骤：(1) 选出 evidence 对应的行；(2) 将选出行的概率除以它们的总和。

$$P(a|b) = \frac{P(a, b)}{P(b)} \quad (\text{Select} + \text{Normalize})$$

Lecture 17: Probabilistic Reasoning II —Bayes Net Representation

本讲核心问题：如何紧凑地表示高维联合概率分布？

88 联合分布的参数爆炸

n 个二值变量的完整联合分布需要 $2^n - 1$ 个独立参数。

例：5 个二值变量 $\rightarrow$ 31 个参数。太多！

解决方案：利用条件独立性简化 $\rightarrow$ Bayesian Network。

89 Bayesian Network (贝叶斯网络)

定义：BN = DAG (有向无环图) + CPTs (条件概率表)。

语义：每个节点的 CPT 只依赖其父节点。

联合分布分解：

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{parents}(X_i))$$

参数节省示例 (Alarm Network)：

- Full joint: $2^5 - 1 = 31$ 个参数
- BN: 只需 $1 + 1 + 4 + 2 + 2 = 10$ 个参数

90 D-Separation (方向分离)

D-Separation 是判断 BN 中条件独立性的图形化方法。

三种基本结构：

(1) Chain (链): $A \rightarrow B \rightarrow C$

- 无条件: A 与 C 不独立
- 给定 B : $A \perp C \mid B$ (B 阻断信息流)

(2) Fork (分叉): $A \leftarrow B \rightarrow C$

- 无条件: A 与 C 不独立 (B 是公共原因)
- 给定 B : $A \perp C \mid B$ (B 阻断)

(3) Collider / V-structure (对撞): $A \rightarrow B \leftarrow C$

- 无条件: $A \perp C$ (B 阻断)
- 给定 B (或 B 的后代): A 与 C 不再独立 (B 被激活, "Explaining Away")

高频考点! 给一个 BN 结构, 判断某对变量是否条件独立。

方法: 检查所有连接路径上的节点:

(1) Chain/Fork 中间节点被 observe $\rightarrow$ 阻断;

(2) Collider 节点未被 observe $\rightarrow$ 阻断; 被 observe $\rightarrow$ 激活。

所有路径都被阻断 $\Leftrightarrow$ 条件独立。

91 BN 的构建原则

选择节点顺序: 通常按因果顺序排列 (原因 $\rightarrow$ 结果)。

添加边的规则: 只有当 X_i 不能被其前面的变量完全解释时, 才需要加边到其父节点。

关键性质: 不同的节点顺序可能导致不同结构 (但编码相同的独立性关系)。因果顺序通常产生最简洁的 BN。

$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{parents}(X_i))$$

Lecture 18: Probabilistic Reasoning III —Bayes Net Inference

本讲核心问题：给定贝叶斯网络和证据 (evidence)，如何计算查询变量的后验概率？

92 推理问题形式化

给定：

- BN 结构 + CPTs
- Evidence variables $E = e$ (已观测的变量值)
- Query variable Q

目标：计算 $P(Q \mid E = e)$ 。

93 Inference by Enumeration (枚举推理)

方法：对所有 hidden variables 求和：

$$P(Q \mid e) = \frac{1}{Z} \sum_{h_1} \sum_{h_2} \cdots \prod_i P(X_i \mid \text{parents}(X_i))$$

问题：指数级复杂度——若有 k 个 hidden variables，每个取 d 值，需要 d^k 次运算。

94 Variable Elimination (VE) (变量消除)

核心思想：交替执行 Join (合并因子) 和 Eliminate (消去变量)，避免构建完整联合分布。

算法步骤：

1. **初始化：**将每个 CPT 作为一个 factor。将 evidence 代入对应 factor (固定 evidence 变量的值)。
2. **对每个 hidden variable H ：**
 - Join：将所有包含 H 的 factors 合并为一个大 factor (pointwise product)
 - Eliminate：对 H 求和 (sum out)，得到不含 H 的新 factor
3. **最终：**将剩余 factors 合并并归一化 $\rightarrow$ 得到 $P(Q \mid e)$ 。

95 Factor 操作详解

Join (合并 / Pointwise Product):

将两个 factor 按公共变量逐行相乘。

示例: $f_1(A, B)$ 和 $f_2(B, C)$ 合并为 $f_3(A, B, C)$, 其中 $f_3(a, b, c) = f_1(a, b) \times f_2(b, c)$ 。

Eliminate (消去 / Sum Out):

对某变量求和消去。

示例: $f(A, B)$ 消去 B 得到 $g(A)$, 其中 $g(a) = \sum_b f(a, b)$ 。

96 VE 计算示例

BN 结构: $A \rightarrow B \rightarrow C$, 查询 $P(C \mid A = 1)$ 。

给定 CPTs: $P(A)$, $P(B|A)$, $P(C|B)$ 。

步骤:

1. 初始 factors: $f_1(A) = P(A = 1)$ (evidence 已代入), $f_2(A, B) = P(B|A = 1)$ (代入 $A = 1$ 后为 $f_2(B)$), $f_3(B, C) = P(C|B)$ 。
2. Hidden variable: B .
 - Join $f_2(B)$ 和 $f_3(B, C)$: 得到 $f_4(B, C) = f_2(B) \times f_3(B, C)$ 。
 - Eliminate B : $f_5(C) = \sum_B f_4(B, C)$ 。
3. Normalize $f_5(C) \rightarrow P(C|A = 1)$ 。

97 Elimination Order (消去顺序)

消去顺序影响效率: 决定了中间 factor 的最大 size。

最优消去顺序: NP-hard 问题。实践中用启发式 (如 min-degree, min-fill)。

关键点: VE 的结果与消去顺序无关 (都正确), 但效率差异巨大。

计算题: 给定一个小型 BN 和 CPTs, 要求用 VE 计算某个条件概率。

步骤必须写清: (1) 列出初始 factors; (2) 代入 evidence; (3) 对每个 hidden variable 做 Join + Eliminate; (4) 最终归一化。

VE = 逐变量 Join + Eliminate, 避免构建完整联合分布

Lecture 19: Naive Bayes Classifier

本讲核心问题：如何用概率模型做分类？如何处理”特征太多”的问题？

98 Naive Bayes 的核心假设

条件独立假设：给定类别 Y ，所有特征 F_i 之间条件独立：

$$P(F_1, F_2, \dots, F_n | Y) = \prod_{i=1}^n P(F_i | Y)$$

为什么”Naive”？这个假设通常不成立（特征之间有相关性），但实践中效果仍然不错。

为什么有用？参数数量从指数级 $O(d^n)$ 降到线性 $O(nd)$ 。

99 分类规则

Bayes Rule 应用于分类：

$$P(Y | F_1, \dots, F_n) \propto P(Y) \prod_{i=1}^n P(F_i | Y)$$

预测：

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(f_i | y)$$

直觉：对每个候选类别 y ，计算”先验 $\times$ 各特征的似然”，取最大者。

100 参数估计

Maximum Likelihood Estimation (MLE)：

$$P(F_i = f | Y = y) = \frac{\text{count}(F_i = f, Y = y)}{\text{count}(Y = y)}$$

问题：如果某个特征值从未在某类别中出现过， $P(F_i = f | y) = 0$ ，导致整个乘积为 0！

101 Laplace Smoothing（拉普拉斯平滑）

解决零概率问题：给每个计数加一个”虚拟样本”：

$$P_{LAP}(F_i = f | Y = y) = \frac{\text{count}(F_i = f, Y = y) + k}{\text{count}(Y = y) + k \cdot |V_i|}$$

逐符号解释：

- k : 平滑参数 (通常 $k = 1$)
- $|V_i|$: 特征 F_i 的取值数量 (vocabulary size)
- 效果: 每个取值至少有 k 次”伪计数”

102 Naive Bayes 完整计算示例

题设: 分类垃圾邮件 (Spam/Ham)。词汇 $\{buy, free, hello\}$ 。

训练数据统计 (已含 Laplace smoothing $k = 1$, $|V| = 3$):

	$P(Y)$	$P(buy Y)$	$P(free Y)$	$P(hello Y)$
Spam	0.4	0.5	0.4	0.1
Ham	0.6	0.1	0.1	0.8

测试邮件含词: buy, free。

$$\begin{aligned}
 P(\text{Spam} \mid \text{buy}, \text{free}) &\propto P(\text{Spam}) \cdot P(\text{buy}|\text{Spam}) \cdot P(\text{free}|\text{Spam}) \\
 &= 0.4 \times 0.5 \times 0.4 = 0.08
 \end{aligned}$$

$$\begin{aligned}
 P(\text{Ham} \mid \text{buy}, \text{free}) &\propto P(\text{Ham}) \cdot P(\text{buy}|\text{Ham}) \cdot P(\text{free}|\text{Ham}) \\
 &= 0.6 \times 0.1 \times 0.1 = 0.006
 \end{aligned}$$

归一化: $P(\text{Spam}|\text{buy}, \text{free}) = \frac{0.08}{0.08+0.006} \approx 0.93$

结论: 预测为 Spam。

计算题可能给出词频统计, 要求: (1) 用 Laplace smoothing 计算 CPT; (2) 对新样本进行分类。

$$\hat{y} = \arg \max_y P(y) \prod_i P(f_i \mid y)$$

Lecture 20: Logic Reasoning

本讲核心问题：如何用形式化逻辑表达知识并进行推理？

103 Propositional Logic (命题逻辑)

基本连接词：

- $\neg P$: 非 (NOT)
- $P \wedge Q$: 与 (AND)
- $P \vee Q$: 或 (OR)
- $P \rightarrow Q$: 蕴含 (IF...THEN)
- $P \leftrightarrow Q$: 双条件 (IFF)

关键等价关系：

$$\begin{aligned}P \rightarrow Q &\equiv \neg P \vee Q \\ \neg(P \wedge Q) &\equiv \neg P \vee \neg Q \quad (\text{De Morgan}) \\ \neg(P \vee Q) &\equiv \neg P \wedge \neg Q \quad (\text{De Morgan})\end{aligned}$$

局限：命题逻辑只能表达固定数量的事实，无法表达”所有””存在”等量化关系。

104 First-Order Logic (FOL) (一阶逻辑)

新增元素：

- 变量: x, y, z (代表对象)
- 谓词: $P(x), Loves(x, y)$ (描述对象的性质或关系)
- 量词: $\forall$ (for all)、 $\exists$ (there exists)
- 函数: $father(x)$ (返回对象的函数)

105 FOL 翻译的核心规则

这是样卷大题！必须牢记以下搭配规则。

最关键的规则：

- $\forall$ 配 $\rightarrow$ （全称量词 + 蕴含）
- $\exists$ 配 $\wedge$ （存在量词 + 合取）

标准翻译模板：

自然语言	FOL 翻译
“Every A is B”	$\forall x (A(x) \rightarrow B(x))$
“Some A is B”	$\exists x (A(x) \wedge B(x))$
“No A is B”	$\forall x (A(x) \rightarrow \neg B(x))$
“Only A are B”	$\forall x (B(x) \rightarrow A(x))$
“Every A has a B”	$\forall x (A(x) \rightarrow \exists y (B(y) \wedge \dots))$
“There is a B that every A ...”	$\exists x (B(x) \wedge \forall y (A(y) \rightarrow \dots))$

106 量词的否定

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$
$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

直觉：“不是所有人都满足 P” = “存在某人不满足 P”。

107 常见错误分析

错误 1： $\exists$ 后接 $\rightarrow$ 。

$\exists x (A(x) \rightarrow B(x))$ 等价于 $\exists x (\neg A(x) \vee B(x))$ 。只要存在一个非 A 的对象就为真——几乎永真。

错误 2： $\forall$ 后接 $\wedge$ 。

$\forall x (A(x) \wedge B(x))$ 要求宇宙中所有对象都同时是 A 和 B——太强。

错误 3：量词顺序错误。

$\forall x \exists y P(x, y)$ ：对每个 x 存在一个（可能不同的） y 。

$\exists y \forall x P(x, y)$ ：存在一个固定的 y 对所有 x 都成立。

后者强于前者。

108 FOL Inference 简述

Modus Ponens (肯定前件): 若 P 为真且 $P \rightarrow Q$ 为真, 则 Q 为真。

Universal Instantiation (全称实例化): $\forall x P(x)$ 为真 $\rightarrow P(A)$ 对任意具体对象 A 为真。

Existential Instantiation (存在实例化): $\exists x P(x)$ 为真 $\rightarrow$ 引入新常量 c 使得 $P(c)$ 为真。

考试以翻译为主。推理规则可能作为概念题出现。

$\forall$ 配 $\rightarrow$, $\exists$ 配 $\wedge$ ——最高频考点
--

Lecture 21: Applications — Visual Segmentation

本讲以概念为主，计算题概率低。重点理解各方法的区别和关键模块。

109 语义分割 (Semantic Segmentation)

任务：对图像每个像素预测类别标签。不区分同一类别的不同实例。

核心模型——FCN (Fully Convolutional Network)：

- **Encoder (编码器)：**下采样 (downsample)，提取语义特征，分辨率降低
- **Decoder (解码器)：**上采样 (upsample)，恢复空间分辨率
- **Skip Connections (跳跃连接)：**将 encoder 的高分辨率特征传给 decoder，改善边界精度

110 上采样方法

方法	特点	是否可学习
Nearest Neighbor	复制最近值	否
Max Unpooling	恢复 max pooling 的位置	否
Transpose Convolution	卷积的转置操作	是

Dilated / Atrous Convolution (空洞卷积)：在 kernel 元素间插入空洞，扩大感受野而不增加参数。

111 评估指标——mIoU

$$\text{IoU}(\text{class } c) = \frac{TP_c}{TP_c + FP_c + FN_c}$$

$$\text{mIoU} = \frac{1}{C} \sum_{c=1}^C \text{IoU}(c)$$

112 实例分割 (Instance Segmentation)

任务：不仅预测每个像素的类别，还区分同一类别的不同实例。

核心模型——Mask R-CNN：Faster R-CNN + Mask Head。

- **RoI Align：**使用双线性插值 (bilinear interpolation) 代替量化，保持空间精度。

- **Mask Head**: 对每个检测到的实例预测一个二值 mask。

概念题: “Semantic Segmentation 和 Instance Segmentation 的区别?”

答: Semantic 只分类别不分实例; Instance 同时区分类别和实例。

Lecture 22: Applications —Object Detection

本讲以概念为主，可能考 IoU 计算和 *Anchor* 概念。

113 Object Detection 任务

输入：图像。输出：每个目标的 bounding box (x, y, w, h) + 类别标签 + 置信度。

114 Two-Stage Detectors（两阶段检测器）

R-CNN 系列：

- **R-CNN**：Selective Search 提取约 2000 个 region proposals $\rightarrow$ 每个单独过 CNN $\rightarrow$ SVM 分类。缺点：极慢。
- **Fast R-CNN**：整图过 CNN 得 feature map $\rightarrow$ RoI Pooling 提取每个 proposal 的特征 $\rightarrow$ FC 层分类 + 回归。
- **Faster R-CNN**：用 Region Proposal Network (RPN) 替代 Selective Search。端到端训练。

Anchor Boxes (锚框)：在 feature map 每个位置预定义多种 scale 和 aspect ratio 的参考框。RPN 对每个 anchor 预测前景/背景 + 边框偏移。

115 One-Stage Detectors（单阶段检测器）

- **YOLO**：图像分为 $S \times S$ 网格，每格预测 B 个 bbox + C 类概率。一次前向传播完成检测，速度极快。
- **SSD**：多尺度 feature maps 上检测（小物体用高分辨率层，大物体用低分辨率层）。

Two-stage vs One-stage：

	Two-Stage	One-Stage
速度	较慢	较快
精度	通常较高	略低（但差距在缩小）
代表	Faster R-CNN	YOLO, SSD

116 关键概念

IoU (Intersection over Union):

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

常用阈值: $\text{IoU} \geq 0.5$ 判为正确检测。

Non-Maximum Suppression (NMS): 对同一物体的多个检测框, 只保留置信度最高的, 抑制 $\text{IoU} > \text{threshold}$ 的重叠框。

mAP (mean Average Precision): 对每类计算 PR 曲线下面积 (AP), 取所有类别平均。

可能的计算题: 给两个 bbox 的坐标 (左上角 + 右下角), 计算 IoU。

步骤: (1) 计算交集面积; (2) 计算并集面积 = 面积 A + 面积 B - 交集; (3) $\text{IoU} = \text{交集} / \text{并集}$ 。

Lecture 23: Self-Supervised Representation Learning

本讲以概念为主，计算题概率极低。

117 Self-Supervised Learning 概述

核心思想：不使用人工标注，利用数据本身的结构创造监督信号。

流程：设计 pretext task 训练 $\rightarrow$ 学到好的 representation $\rightarrow$ 迁移到 downstream task。

118 Pretext Tasks (代理任务)

- **Rotation Prediction:** 预测图像被旋转了多少度 ($0^\circ/90^\circ/180^\circ/270^\circ$, 4 分类)
- **Jigsaw Puzzle:** 打乱 patch 顺序, 预测正确排列
- **Inpainting:** 遮挡一部分, 预测被遮挡内容
- **Colorization:** 给灰度图上色

119 Contrastive Learning (对比学习)

核心思想：拉近正样本对 (positive pairs), 推远负样本对 (negative pairs)。

InfoNCE Loss: N -way classification, 从 $N - 1$ 个负样本中识别出正样本。

SimCLR 框架：

1. 对同一图像做两次不同的数据增强 $\rightarrow$ 正样本对
2. Encoder $\rightarrow$ Projection Head $\rightarrow$ 得到 embedding
3. 用 cosine similarity + contrastive loss 训练
4. 需要大 batch size (更多负样本)

120 CLIP (Contrastive Language-Image Pre-training)

思想：跨模态对比学习——图像和文本配对。

训练： N 张图 + N 段文本构成 $N \times N$ 的 affinity matrix。最大化对角线 (正确配对) 的相似度。

Zero-shot 能力：用 text prompt（如“A photo of a cat”）构造分类器，无需 fine-tuning。

概念题：“Self-supervised learning 与 supervised learning 的主要区别？”

答：Self-supervised 不需要人工标注，用数据自身结构生成监督信号。

Lecture 24: Visual-Language Pretraining for Open World

本讲以概念为主, *Depthwise Convolution* 参数计算可能考。

121 Open World Problems

挑战:

- Dataset bias / domain shift
- Novel categories (训练时未见过的类别)
- Long-tail distribution (少数类样本极少)

122 CLIP 应用于 Open World

Zero-shot Classification:

1. 构造 text prompts: “A photo of a {class}”
2. Text Encoder $\rightarrow$ text features
3. Image Encoder $\rightarrow$ image feature
4. 预测 = cosine similarity 最大的 class

Open World Detection (ViLD):

- 训练时: RPN proposals $\rightarrow$ Region features R_i 。用 CLIP Image Encoder 产生 R'_i 作为教师信号 (Knowledge Distillation)。
- 推理时: 只用 R_i 与 text features (base + novel 类别) 比较。

Open World Segmentation (LSeg):

- Image Encoder (ViT) $\rightarrow$ pixel features ($\tilde{H} \times \tilde{W} \times C$)
- Text Encoder $\rightarrow N$ 个类别的 text features ($N \times C$)
- 矩阵乘法 $\rightarrow$ affinity map ($\tilde{H} \times \tilde{W} \times N$)
- Depthwise conv + upsample $\rightarrow$ 分割结果

123 Depthwise Separable Convolution (深度可分离卷积)

标准卷积参数量:

$$\text{Params}_{\text{standard}} = k^2 \times C_{\text{in}} \times C_{\text{out}}$$

Depthwise Separable 卷积参数量:

$$\text{Params}_{\text{DW}} = \underbrace{k^2 \times C_{\text{in}}}_{\text{Depthwise}} + \underbrace{1 \times 1 \times C_{\text{in}} \times C_{\text{out}}}_{\text{Pointwise}}$$

两步分解:

- **Depthwise Conv**: 每个输入通道单独用一个 $k \times k$ kernel 卷积 (不混合通道)
- **Pointwise Conv**: 1×1 卷积混合通道

计算示例: Input 3 channels, 3×3 kernel, output 5 channels。

$$\text{Standard: } 3 \times 3 \times 3 \times 5 = 135$$

$$\text{Depthwise Separable: } 3 \times 3 \times 3 + 1 \times 1 \times 3 \times 5 = 27 + 15 = 42$$

参数节省比例: $42/135 \approx 31\%$ 。

计算题: 给定 C_{in} , C_{out} , kernel size k , 计算标准卷积和 depthwise separable 卷积的参数量。

DW-Separable: $k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}} \ll k^2 C_{\text{in}} C_{\text{out}}$

A 答题模板 (Answer Templates)

A.1 Problem 1 类型: First-Order Logic 翻译

A.1.1 题型识别

题目形式: 给定若干一元/二元谓词, 将自然语言句子翻译为一阶逻辑公式。

A.1.2 核心规则速记

1. “Some A are B” $\Rightarrow \exists x (A(x) \wedge B(x))$
 - 存在量词 + 合取 ($\wedge$), 不要用 $\rightarrow$ 。
2. “Every A does B” / “All A are B” $\Rightarrow \forall x (A(x) \rightarrow B(x))$
 - 全称量词 + 蕴含 ($\rightarrow$), 不要用 $\wedge$ 。
3. “There is a B that every A ...” $\Rightarrow \exists x (B(x) \wedge \forall y (A(y) \rightarrow \dots))$
 - 先存在再全称, 注意量词顺序。
4. “Every A has a B (failed/passed/...)” $\Rightarrow \forall x (A(x) \rightarrow \exists y (B(y) \wedge \dots))$
 - 先全称再存在。

A.1.3 样卷题目复现与答题步骤

给定谓词: $\text{Teacher}(x)$, $\text{Student}(x)$, $\text{Test}(x)$, $\text{Passed}(x, y)$

(i) “Some students are also teachers.”

1. 识别关键词 “Some ...are ...” $\rightarrow$ 存在量词 + 合取
2. 写出: $\exists x (\text{Student}(x) \wedge \text{Teacher}(x))$

(ii) “Every student has a test failed.”

1. “Every student” $\rightarrow \forall x (\text{Student}(x) \rightarrow \dots)$
2. “has a test” $\rightarrow \exists y (\text{Test}(y) \wedge \dots)$
3. “failed” = 没通过 $\rightarrow \neg \text{Passed}(x, y)$
4. 组合: $\forall x (\text{Student}(x) \rightarrow \exists y (\text{Test}(y) \wedge \neg \text{Passed}(x, y)))$

(iii) “There is a test that every student has passed.”

1. “There is a test” $\rightarrow \exists x (\text{Test}(x) \wedge \dots)$
2. “every student” $\rightarrow \forall y (\text{Student}(y) \rightarrow \dots)$
3. “has passed” $\rightarrow \text{Passed}(y, x)$
4. 组合: $\exists x (\text{Test}(x) \wedge \forall y (\text{Student}(y) \rightarrow \text{Passed}(y, x)))$

A.1.4 常见错误

- **错误 1:** “Some A are B” 写成 $\exists x (A(x) \rightarrow B(x))$ — 这在逻辑上几乎永真。
- **错误 2:** “Every A does B” 写成 $\forall x (A(x) \wedge B(x))$ — 这要求所有事物都是 A 且 B。
- **错误 3:** 量词作用域不对, 如 $\forall x \exists y$ 写反成 $\exists y \forall x$ (语义完全不同)。
- **错误 4:** $\text{Passed}(x, y)$ 中 x, y 顺序颠倒 — 注意谓词定义 “ x passed y ”。

A.2 Problem 2 类型: Markov Decision Process (MDP)

A.2.1 题型识别

题目形式: 给定 Gridworld (网格世界), 含终端状态、奖励、转移概率, 要求计算:

1. 期望即时奖励 $r(s, a)$
2. Value Iteration 迭代值 $V_k(s)$
3. 最优策略 $\pi^*(s)$

A.2.2 Part (i): 计算期望即时奖励

公式:

$$r(s, a) = \sum_{s'} T(s, a, s') R(s, a, s')$$

答题步骤:

1. 确定当前状态 s 和动作 a 。
2. 列出所有可能的下一状态 s' (含撞墙弹回)。
3. 对每个 s' : 转移概率 $T(s, a, s') \times$ 进入 s' 获得的奖励 $R(s, a, s')$ 。
4. 求和。

样卷示例: 状态 (2,2), 动作 East

- 意图方向 East $\rightarrow$ 到达 (2,3): 概率 0.8, 奖励 +5 $\rightarrow$ 贡献 $0.8 \times 5 = 4.0$
- 垂直方向 North $\rightarrow$ 到达 (2,2) 撞上方墙弹回: 概率 0.1, 奖励 0 $\rightarrow$ 贡献 $0.1 \times 0 = 0$
- 垂直方向 South $\rightarrow$ 到达 (1,2): 概率 0.1, 奖励 0 $\rightarrow$ 贡献 $0.1 \times 0 = 0$
- $r((2,2), E) = 4.0 + 0 + 0 = 4.00$

样卷示例: 状态 (1,2), 动作 East

- 意图方向 East $\rightarrow$ 到达 (1,3): 概率 0.8, 奖励 -5 $\rightarrow$ 贡献 $0.8 \times (-5) = -4.0$
- 垂直方向 North $\rightarrow$ 到达 (2,2): 概率 0.1, 奖励 0 $\rightarrow$ 贡献 0
- 垂直方向 South $\rightarrow$ 到达 (1,2) 撞墙弹回: 概率 0.1, 奖励 0 $\rightarrow$ 贡献 0
- $r((1,2), E) = -4.0$

关键点:

- 只有进入终端状态时 $R \neq 0$, 进入非终端状态 $R = 0$ 。
- 撞墙 = 留在原地, 但奖励仍为 0 (因为不是“进入”终端状态)。
- 终端状态本身没有出边动作 (表格填 0)。

A.2.3 Part (ii): Value Iteration 计算

公式 (Bellman Update):

$$V_{i+1}(s) = \max_a \left[\sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V_i(s')) \right]$$

等价写法:

$$V_{i+1}(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} T(s, a, s') V_i(s') \right]$$

答题步骤 (以 $V_2(s = (2, 1))$ 为例):

1. 写出公式: $V_2((2, 1)) = \max_a q_1((2, 1), a)$
2. 对每个动作 a 计算 $q_1((2, 1), a)$:

$$\begin{aligned} q_1((2, 1), E) &= 0.9 \times (0.8 \times V_1((2, 2)) + 0.1 \times V_1((1, 1)) + 0.1 \times V_1((2, 1))) \\ &= 0.9 \times (0.8 \times 4 + 0.1 \times 0 + 0.1 \times 0) = 0.9 \times 3.2 = 2.88 \end{aligned}$$

$$\begin{aligned} q_1((2, 1), S) &= 0.9 \times (0.8 \times V_1((1, 1)) + 0.1 \times V_1((2, 2)) + 0.1 \times V_1((2, 1))) \\ &= 0.9 \times (0 + 0.4 + 0) = 0.36 \end{aligned}$$

$$\begin{aligned} q_1((2, 1), W) &= 0.9 \times (0.8 \times V_1((2, 1)) + 0.1 \times V_1((2, 1)) + 0.1 \times V_1((1, 1))) \\ &= 0.9 \times 0 = 0 \end{aligned}$$

$$\begin{aligned} q_1((2, 1), N) &= 0.9 \times (0.8 \times V_1((2, 1)) + 0.1 \times V_1((2, 1)) + 0.1 \times V_1((2, 2))) \\ &= 0.9 \times (0 + 0 + 0.4) = 0.36 \end{aligned}$$

3. 取最大值: $V_2((2, 1)) = \max(2.88, 0.36, 0, 0.36) = 2.88$

关键点:

- 终端状态的 V 永远为 0 (没有后续动作)。
- 注意 γ 的位置 — 样卷公式形式为 $T(R + \gamma V)$ 。当非终端状态 $R = 0$ 时简化为 $\gamma \cdot T \cdot V_i(s')$ 。
- 注意方向与邻居的对应关系 (样卷中 North = 行号增大方向, 即从第 1 行到第 2 行)。

A.2.4 Part (iii): 最优策略提取

公式:

$$\pi_k^*(s) = \arg \max_a \left[\sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V_k(s')) \right]$$

答题步骤 (以 $\pi_2^*(s = (1, 1))$ 为例):

1. 写出公式。
2. 对每个动作计算 $q_2((1, 1), a)$, 使用 $V_2(s)$ 的值:

$$q_2((1, 1), E) = 0.9 \times (0.8 \times 2.38 + 0.1 \times 0 + 0.1 \times 2.88) = 1.97$$

$$q_2((1, 1), S) = 0.9 \times (0.8 \times 0 + 0.1 \times 0 + 0.1 \times 2.38) = 0.21$$

$$q_2((1, 1), W) = 0.9 \times (0.8 \times 0 + 0.1 \times 2.88 + 0.1 \times 0) = 0.26$$

$$q_2((1, 1), N) = 0.9 \times (0.8 \times 2.88 + 0.1 \times 2.38 + 0.1 \times 0) = 2.29$$

3. 取 argmax: 最大值 2.29 对应 North。

4. 结论: $\pi_2^*((1, 1)) = N$

关键点：

- 策略提取与 Value Iteration 计算步骤几乎相同，只是取 argmax 而非 max 。
- 注意使用的是 V_k 而非 V_{k-1} （题目会明确指定用哪轮的值）。
- 终端状态标记 NA（无需决策）。

A.2.5 Gridworld 方向辨认（样卷坐标系）

(2,1)	(2,2)	(2,3) [+5]
(1,1) S	(1,2)	(1,3) [-5]

- **North**：行号增大（从第 1 行到第 2 行，即向上）
- **South**：行号减小（从第 2 行到第 1 行，即向下）
- **East**：列号增大（向右）
- **West**：列号减小（向左）
- **垂直方向**：若意图 North/South，垂直为 East/West；若意图 East/West，垂直为 North/South

B 关键词识别与考点速查**B.1 FOL 翻译关键词对照表**

自然语言关键词	逻辑结构	注意事项
Some / There exists	$\exists x$	后接 $\wedge$ （不是 $\rightarrow$ ）
Every / All / Each	$\forall x$	后接 $\rightarrow$ （不是 $\wedge$ ）
No / None	$\forall x (A(x) \rightarrow \neg B(x))$ 或 $\neg \exists x (A \wedge B)$	两种写法等价
... has a ...	嵌套 $\exists y$	在 $\rightarrow$ 右侧
... that every ...	嵌套 $\forall y$	在 $\wedge$ 右侧
not / failed / didn't	$\neg$	确认否定作用于哪个谓词
Only	$\forall x (B(x) \rightarrow A(x))$	“Only A are B”

B.2 MDP 计算步骤速查

题目要求	所用公式	关键操作
期望即时奖励 $r(s, a)$	$\sum_{s'} T(s, a, s') R(s, a, s')$	列出 3 个可能 s' , 乘奖励求和
Value Iteration	$V_{i+1}(s) = \max_a [\dots]$	对每个动作算 q , 取 $\max$
最优策略提取	$\pi^*(s) = \arg \max_a [\dots]$	同上, 但取 $\arg \max$
Q-value	$q_i(s, a) = \sum_{s'} T(\cdot)(R + \gamma V_i(s'))$	展开转移函数

C 核心公式速查表

C.1 First-Order Logic

$$\text{存在量化 (Some/There is) : } \exists x (P(x) \wedge Q(x)) \quad (1)$$

$$\text{全称量化 (Every/All) : } \forall x (P(x) \rightarrow Q(x)) \quad (2)$$

$$\text{否定存在 (No/None) : } \forall x (P(x) \rightarrow \neg Q(x)) \equiv \neg \exists x (P(x) \wedge Q(x)) \quad (3)$$

$$\text{嵌套量词 (外 } \forall \text{ 内 } \exists \text{) : } \forall x (A(x) \rightarrow \exists y (B(y) \wedge C(x, y))) \quad (4)$$

$$\text{嵌套量词 (外 } \exists \text{ 内 } \forall \text{) : } \exists x (B(x) \wedge \forall y (A(y) \rightarrow C(y, x))) \quad (5)$$

C.2 Markov Decision Process

C.2.1 基本定义

$$\text{期望即时奖励: } r(s, a) = \sum_{s'} T(s, a, s') R(s, a, s') \quad (6)$$

$$\text{状态值函数: } V^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s, \pi \right] \quad (7)$$

$$\text{Q 值函数: } Q^\pi(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^\pi(s')] \quad (8)$$

C.2.2 Value Iteration

$$V_{i+1}(s) = \max_a \left[\sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V_i(s')) \right] \quad (9)$$

$$= \max_a \left[r(s, a) + \gamma \sum_{s'} T(s, a, s') V_i(s') \right] \quad (10)$$

C.2.3 最优策略提取

$$\pi^*(s) = \arg \max_a \left[\sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V^*(s')) \right] \quad (11)$$

C.2.4 Gridworld 转移概率模板

对于 intended action a ，设到达目标邻居为 s_{front} ，两侧邻居为 $s_{\text{left}}, s_{\text{right}}$ ：

$$T(s, a, s_{\text{front}}) = 0.8 \quad (\text{若撞墙则概率加回 } s \text{ 本身}) \quad (12)$$

$$T(s, a, s_{\text{left}}) = 0.1 \quad (\text{若撞墙则概率加回 } s \text{ 本身}) \quad (13)$$

$$T(s, a, s_{\text{right}}) = 0.1 \quad (\text{若撞墙则概率加回 } s \text{ 本身}) \quad (14)$$

D 易错点与考试提醒

D.1 FOL 翻译

1. **$\exists$ 配 $\wedge, \forall$ 配 $\rightarrow \neg$** 这是最高频错误，务必牢记。
2. **量词顺序** — “There is a test that every student passed” 中， $\exists$ 在外， $\forall$ 在内。如果写反，语义变成“每个学生都存在一个（可能不同的）test 被通过”。
3. **谓词参数顺序** — $\text{Passed}(x, y)$ 表示“ x passed y ”，写成 $\text{Passed}(y, x)$ 意思完全不同。
4. **否定词位置** — “has a test failed” = $\neg \text{Passed}(x, y)$ ，否定作用于 Passed ，而非 Test 。

D.2 MDP 计算

1. **方向确认** — 拿到题目第一件事：标清 North/South/East/West 分别对应行列号增减的哪个方向。样卷中 North = 行号 +1。

2. **撞墙处理** — 撞墙不是“消失”，而是留在原地。概率不变（仍是 0.8 或 0.1），只是 $s' = s$ 。
3. **终端状态** — $V(\text{terminal}) = 0$ 永远不变，不参与 Value Iteration 更新。
4. **γ 的位置** — 样卷公式中 γ 在 $V_i(s')$ 前面， R 不乘 γ : $T(R + \gamma V)$ 。不要把 γ 也乘到 R 上。
5. **非终端状态 $R = 0$ 的简化** — 当 $R(s, a, s') = 0$ （即 s' 非终端），该项退化为 $\gamma T \cdot V_i(s')$ 。但如果 s' 是终端状态，则 $R \neq 0$ 且 $V_i(s') = 0$ ，该项变为 $T \cdot R$ 。
6. **精度** — 题目要求保留两位小数时，中间计算尽量多保留位数，最后再四舍五入。
7. **Policy 提取使用哪轮 V** — 题目可能说“based on V_2 ”，则计算 q 时用 V_2 的值，不是 V_1 。

D.3 考试时间分配建议

题目类型	建议时间	策略
FOL 翻译	10–15 min	套模板，检查 $\exists/\forall$ 搭配
$r(s, a)$ 表格填写	10–15 min	只关注能到达终端状态的 (state, action) 对
Value Iteration 推导	15–20 min	先写公式，再逐动作展开，标清数字
最优策略	10–15 min	复用 VI 的计算结构，取 argmax